One reason for working with the inverses is that we want to factor A, not U. The "inverse form" gives $A = LU$. Another reason is that we get something extra, almost more than we deserve. This is the third point, showing that L is exactly right.

Third point: Each multiplier ℓ_{ij} goes directly into its i, j position—*unchanged*—in the product of inverses which is L. Usually matrix multiplication will mix up all the numbers. Here that doesn't happen. The order is right for the inverse matrices, to keep the ℓ's unchanged. The reason is given below in equation (3).

Since each E^{-1} has 1's down its diagonal, the final good point is that L does too.

($A = LU$) This is elimination without row exchanges. The upper triangular U has the pivots on its diagonal. The lower triangular L has all 1's on its diagonal. ***The multipliers ℓ_{ij} are below the diagonal of L.***

Example 1 Elimination subtracts $\frac{1}{2}$ times row 1 from row 2. The last step subtracts $\frac{2}{3}$ times row 2 from row 3. The lower triangular L has $\ell_{21} = \frac{1}{2}$ and $\ell_{32} = \frac{2}{3}$. Multiplying LU produces A:

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 0 & \frac{2}{3} & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 0 \\ 0 & \frac{3}{2} & 1 \\ 0 & 0 & \frac{4}{3} \end{bmatrix} = LU.$$

The (3, 1) multiplier is zero because the (3, 1) entry in A is zero. No operation needed.

Example 2 Change the top left entry from 2 to 1. The pivots all become 1. The multipliers are all 1. That pattern continues when A is 4 by 4:

Special pattern

$$A = \begin{bmatrix} \mathbf{1} & 1 & 0 & 0 \\ 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & & & \\ 1 & 1 & & \\ 0 & 1 & 1 & \\ 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 & 0 \\ & 1 & 1 & 0 \\ & & 1 & 1 \\ & & & 1 \end{bmatrix}.$$

These LU examples are showing something extra, which is very important in practice. Assume no row exchanges. When can we predict *zeros* in L and U?

When a row of A starts with zeros, so does that row of L.
When a column of A starts with zeros, so does that column of U.

If a row starts with zero, we don't need an elimination step. L has a zero, which saves computer time. Similarly, zeros at the *start* of a column survive into U. But please realize: Zeros in the *middle* of a matrix are likely to be filled in, while elimination sweeps forward. We now explain why L has the multipliers ℓ_{ij} in position, with no mix-up.

The key reason why A equals LU: Ask yourself about the pivot rows that are subtracted from lower rows. Are they the original rows of A? *No*, elimination probably changed them. Are they rows of U? *Yes*, the pivot rows never change again. When computing the third

row of U, we subtract multiples of earlier rows of U (*not rows of* A!):

$$\text{Row 3 of } U = (\text{Row 3 of } A) - \ell_{31}(\text{Row 1 of } U) - \ell_{32}(\text{Row 2 of } U). \tag{2}$$

Rewrite this equation to see that the row $[\ell_{31} \quad \ell_{32} \quad 1]$ is multiplying U:

$$(\text{Row 3 of } A) = \ell_{31}(\text{Row 1 of } U) + \ell_{32}(\text{Row 2 of } U) + 1(\text{Row 3 of } U). \tag{3}$$

This is exactly row 3 ***of*** $A = LU$. That row of L holds $\ell_{31}, \ell_{32}, 1$. All rows look like this, whatever the size of A. With no row exchanges, we have $A = LU$.

Better balance The $L\,U$ factorization is "unsymmetric" because U has the pivots on its diagonal where L has 1's. This is easy to change. ***Divide U by a diagonal matrix D that contains the pivots.*** That leaves a new matrix with 1's on the diagonal:

$$\text{Split } U \text{ into } \begin{bmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{bmatrix} \begin{bmatrix} 1 & u_{12}/d_1 & u_{13}/d_1 & \cdot \\ & 1 & u_{23}/d_2 & \cdot \\ & & \ddots & \vdots \\ & & & 1 \end{bmatrix}.$$

It is convenient (but a little confusing) to keep the same letter U for this new upper triangular matrix. It has 1's on the diagonal (like L). Instead of the normal LU, the new form has D in the middle: ***Lower triangular L times diagonal D times upper triangular U.***

The triangular factorization can be written $A = LU$ ***or*** $A = LDU$.

Whenever you see LDU, it is understood that U has 1's on the diagonal. *Each row is divided by its first nonzero entry—the pivot.* Then L and U are treated evenly in LDU:

$$\begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} 2 & 8 \\ 0 & 5 \end{bmatrix} \quad \text{splits further into} \quad \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} 2 & \\ & 5 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 0 & 1 \end{bmatrix}. \tag{4}$$

The pivots 2 and 5 went into D. Dividing the rows by 2 and 5 left the rows $[1 \quad 4]$ and $[0 \quad 1]$ in the new U with diagonal ones. The multiplier 3 is still in L.

My own lectures sometimes stop at this point. The next paragraphs show how elimination codes are organized, and how long they take. If MATLAB (or any software) is available, you can measure the computing time by just counting the seconds.

One Square System = Two Triangular Systems

The matrix L contains our memory of Gaussian elimination. It holds the numbers that multiplied the pivot rows, before subtracting them from lower rows. When do we need this record and how do we use it in solving $A\boldsymbol{x} = \boldsymbol{b}$?

We need L as soon as there is a *right side* $\boldsymbol{b}$. The factors L and U were completely decided by the left side (the matrix A). On the right side of $A\boldsymbol{x} = \boldsymbol{b}$, we use L^{-1} and then U^{-1}. That ***Solve*** step deals with two triangular matrices.

1 ***Factor*** (into L and U, by elimination on the left side matrix A)

2 ***Solve*** (forward elimination on $\boldsymbol{b}$ using L, then back substitution for $\boldsymbol{x}$ using U).

Earlier, we worked on A and $\boldsymbol{b}$ at the same time. No problem with that—just augment to $[A \;\; \boldsymbol{b}]$. But most computer codes keep the two sides separate. The memory of elimination is held in L and U, to process $\boldsymbol{b}$ whenever we want to. The User's Guide to LAPACK remarks that "This situation is so common and the savings are so important that no provision has been made for solving a single system with just one subroutine."

How does ***Solve*** work on $\boldsymbol{b}$? First, apply forward elimination to the right side (the multipliers are stored in L, use them now). This changes $\boldsymbol{b}$ to a new right side $\boldsymbol{c}$. *We are really solving* $Lc = b$. Then back substitution solves $Ux = c$ as always. The original system $Ax = b$ is factored into ***two triangular systems***:

Forward and backward ***Solve*** $Lc = b$ ***and then solve*** $Ux = c$. (5)

To see that $\boldsymbol{x}$ is correct, multiply $Ux = c$ by L. Then $LUx = Lc$ is just $Ax = b$.

To emphasize: There is *nothing new* about those steps. This is exactly what we have done all along. We were really solving the triangular system $Lc = b$ as elimination went forward. Then back substitution produced $\boldsymbol{x}$. An example shows what we actually did.

Example 3 Forward elimination (downward) on $Ax = b$ ends at $Ux = c$:

$$Ax = b \quad \begin{matrix} u + 2v = 5 \\ 4u + 9v = 21 \end{matrix} \quad \text{becomes} \quad \begin{matrix} u + 2v = 5 \\ v = 1 \end{matrix} \quad Ux = c$$

The multiplier was 4, which is saved in L. The right side used it to change 21 to 1:

$Lc = b$ The lower triangular system $\begin{bmatrix} 1 & 0 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} c \end{bmatrix} = \begin{bmatrix} 5 \\ 21 \end{bmatrix}$ gave $c = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$.

$Ux = c$ The upper triangular system $\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$ gives $x = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$.

L and U can go into the n^2 storage locations that originally held A (now forgettable).

The Cost of Elimination

A very practical question is cost—or computing time. We can solve 1000 equations on a PC. What if $n = 100,000$? (*Not if A is dense.*) Large systems come up all the time in scientific computing, where a three-dimensional problem can easily lead to a million unknowns. We can let the calculation run overnight, but we can't leave it for 100 years.

The first stage of elimination, on column 1, produces zeros below the first pivot. To find each new entry below the pivot row requires one multiplication and one subtraction. *We will count this first stage as n^2 multiplications and n^2 subtractions.* It is actually less, $n^2 - n$, because row 1 does not change.

The next stage clears out the second column below the second pivot. The working matrix is now of size $n-1$. Estimate this stage by $(n-1)^2$ multiplications and subtractions. The matrices are getting smaller as elimination goes forward. The rough count to reach U is the sum of squares $n^2 + (n-1)^2 + \cdots + 2^2 + 1^2$.

There is an exact formula $\frac{1}{3}n(n+\frac{1}{2})(n+1)$ for this sum of squares. When n is large, the $\frac{1}{2}$ and the 1 are not important. *The number that matters is* $\frac{1}{3}n^3$. The sum of squares is like the integral of x^2! The integral from 0 to n is $\frac{1}{3}n^3$:

Elimination on A requires about $\frac{1}{3}n^3$ multiplications and $\frac{1}{3}n^3$ subtractions.

What about the right side $\boldsymbol{b}$? Going forward, we subtract multiples of b_1 from the lower components $b_2, \ldots, b_n$. This is $n-1$ steps. The second stage takes only $n-2$ steps, because b_1 is not involved. The last stage of forward elimination takes one step.

Now start back substitution. Computing x_n uses one step (divide by the last pivot). The next unknown uses two steps. When we reach x_1 it will require n steps ($n-1$ substitutions of the other unknowns, then division by the first pivot). The total count on the right side, from $\boldsymbol{b}$ to $\boldsymbol{c}$ to $\boldsymbol{x}$—*forward to the bottom and back to the top*—is exactly n^2:

$$[(n-1) + (n-2) + \cdots + 1] \; + \; [1 + 2 + \cdots + (n-1) + n] = n^2. \tag{6}$$

To see that sum, pair off $(n-1)$ with 1 and $(n-2)$ with 2. The pairings leave n terms, each equal to n. That makes n^2. The right side costs a lot less than the left side!

Solve ***Each right side needs n^2 multiplications and n^2 subtractions.***

A **band matrix** B has only w nonzero diagonals below and also above its main diagonal. The zero entries outside the band stay zero in elimination (zeros in L and U). Clearing out the first column needs w^2 multiplications and subtractions (w zeros to be produced below the pivot, each one using a pivot row of length w). Then clearing out all n columns, to reach U, needs no more than nw^2. This saves a lot of time:

Band matrices	**Factor**	change $\frac{1}{3}n^3$ to $\boldsymbol{nw^2}$	**Solve**	change n^2 to $\boldsymbol{2nw}$

Here are codes to factor A into LU and to solve $A\boldsymbol{x} = \boldsymbol{b}$. The Teaching code **slu** stops right away if a number smaller than the tolerance "*tol*" appears in a pivot position. The Teaching Codes are on **web.mit.edu/18.06/www**. Professional codes will look down each column for the largest available pivot, to exchange rows and continue solving.

MATLAB's backslash command $\boldsymbol{x} = A\backslash\boldsymbol{b}$ combines **Factor** and **Solve** to reach $\boldsymbol{x}$.

```
function [L, U] = slu(A)
%     Square LU factorization with no row exchanges!
[n, n] = size(A);    tol = 1.e − 6;
for k = 1 : n
  if abs(A(k, k)) < tol
  end       % Cannot proceed without a row exchange: stop
  L(k, k) = 1;
  for i = k + 1 : n
    L(i, k) = A(i, k)/A(k, k);    % Multipliers for column k are put into L
    for j = k + 1 : n    % Elimination beyond row k and column k
      A(i, j) = A(i, j) − L(i, k) ∗ A(k, j);    % Matrix still called A
    end
  end
  for j = k : n
    U(k, j) = A(k, j);        % row k is settled, now name it U
  end
end
function x = slv(A, b)
%     Solve Ax = b using L and U from slu(A).
[L, U] = slu(A); s = 0;    % No row exchanges!
for k = 1 : n    % Forward elimination to solve Lc = b
  for j = 1 : k − 1
    s = s + L(k, j) ∗ c(j);    % Add L times earlier c(j) before c(k)
  end
  c(k) = b(k) − s; s = 0;    % Find c(k) and reset s for next k
end
for k = n : −1 : 1    % Going backwards from x(n) to x(1)
  for j = k + 1 : n    % Back substitution
    t = t + U(k, j) ∗ x(j);    % U times later x(j)
  end
  x(k) = (c(k) − t)/U(k, k);    % Divide by pivot
end
x = x′;    % Transpose to column vector
```

How long does it take to solve $Ax = b$? For a random matrix of order $n = 1000$, a typical time is 1 second. See **web.mit.edu/18.06** and **math.mit.edu/linearalgebra** for the times in MATLAB, Maple, Mathematica, SciLab, Python, and R. The time is multiplied by about 8 when n is multiplied by 2. For professional codes go to **netlib.org**.

According to this n^3 rule, matrices that are 10 times as large (order 10,000) will take a thousand seconds. Matrices of order 100,000 will take a million seconds. This is too expensive without a supercomputer, but remember that these matrices are full. Most matrices in practice are sparse (many zero entries). In that case $A = LU$ is much faster.

For tridiagonal matrices of order 10,000, storing only the nonzeros, solving $Ax = b$ is a breeze. Provided the code recognizes that A is tridiagonal.

■ REVIEW OF THE KEY IDEAS ■

1. Gaussian elimination (with no row exchanges) factors A into L times U.
2. The lower triangular L contains the numbers ℓ_{ij} that multiply pivot rows, going from A to U. The product LU adds those rows back to recover A.
3. On the right side we solve $Lc = b$ (forward) and $Ux = c$ (backward).
4. ***Factor*** : There are $\frac{1}{3}(n^3 - n)$ multiplications and subtractions on the left side.
5. ***Solve*** : There are n^2 multiplications and subtractions on the right side.
6. For a band matrix, change $\frac{1}{3}n^3$ to nw^2 and change n^2 to $2wn$.

■ WORKED EXAMPLES ■

2.6 A The lower triangular Pascal matrix L contains the famous "*Pascal triangle*". Gauss-Jordan found its inverse in the worked example **2.5 C**. This problem connects L to the *symmetric* Pascal matrix P and the upper triangular U. The symmetric P has Pascal's triangle tilted, so each entry is the sum of the entry above and the entry to the left. The n by n symmetric P is pascal(n) in MATLAB.

Problem: *Establish the amazing lower-upper factorization* $P = LU$.

$$\text{pascal(4)} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 1 & 3 & 6 & 10 \\ 1 & 4 & 10 & 20 \end{bmatrix} = \begin{bmatrix} \mathbf{1} & 0 & 0 & 0 \\ \mathbf{1} & \mathbf{1} & 0 & 0 \\ \mathbf{1} & \mathbf{2} & \mathbf{1} & 0 \\ \mathbf{1} & \mathbf{3} & \mathbf{3} & \mathbf{1} \end{bmatrix} \begin{bmatrix} \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} \\ 0 & \mathbf{1} & \mathbf{2} & \mathbf{3} \\ 0 & 0 & \mathbf{1} & \mathbf{3} \\ 0 & 0 & 0 & \mathbf{1} \end{bmatrix} = LU.$$

Then predict and check the next row and column for 5 by 5 Pascal matrices.

Solution You could multiply LU to get P. Better to start with the symmetric P and reach the upper triangular U by elimination:

$$P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 1 & 3 & 6 & 10 \\ 1 & 4 & 10 & 20 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 2 & 5 & 9 \\ 0 & 3 & 9 & 19 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 3 & 10 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix} = U.$$

The multipliers ℓ_{ij} that entered these steps go perfectly into L. Then $P = LU$ is a particularly neat example. *Notice that every pivot is* 1 on the diagonal of U.

The next section will show how symmetry produces a special relationship between the triangular L and U. For Pascal, U is the "**transpose**" of L.

You might expect the MATLAB command lu(pascal(4)) to produce these L and U. That doesn't happen because the **lu** subroutine chooses the largest available pivot in each column. The second pivot will change from 1 to 3. But a "Cholesky factorization" does no row exchanges: U = chol(pascal(4))

The full proof of $P = LU$ for all Pascal sizes is quite fascinating. The paper "*Pascal Matrices*" is on the course web page **web.mit.edu/18.06** which is also available through MIT's *OpenCourseWare* at **ocw.mit.edu**. These Pascal matrices have so many remarkable properties—we will see them again.

2.6 B The problem is: *Solve* $Px = b = (1,0,0,0)$. This right side = column of I means that x will be the first column of P^{-1}. That is Gauss-Jordan, matching the columns of $PP^{-1} = I$. We already know the Pascal matrices L and U as factors of P:

Two triangular systems $Lc = b$ (forward) $Ux = c$ (back).

Solution The lower triangular system $Lc = b$ is solved *top to bottom*:

$$\begin{array}{rl} c_1 & = 1 \\ c_1 + c_2 & = 0 \\ c_1 + 2c_2 + c_3 & = 0 \\ c_1 + 3c_2 + 3c_3 + c_4 & = 0 \end{array} \quad \text{gives} \quad \begin{array}{l} c_1 = +1 \\ c_2 = -1 \\ c_3 = +1 \\ c_4 = -1 \end{array}$$

Forward elimination is multiplication by L^{-1}. It produces the upper triangular system $Ux = c$. The solution x comes as always by back substitution, *bottom to top*:

$$\begin{array}{rl} x_1 + x_2 + x_3 + x_4 & = 1 \\ x_2 + 2x_3 + 3x_4 & = -1 \\ x_3 + 3x_4 & = 1 \\ x_4 & = -1 \end{array} \quad \text{gives} \quad \begin{array}{l} x_1 = +4 \\ x_2 = -6 \\ x_3 = +4 \\ x_4 = -1 \end{array}$$

I see a pattern in that x, but I don't know where it comes from. Try **inv(pascal**(4)).

Problem Set 2.6

Problems 1–14 compute the factorization $A = LU$ (and also $A = LDU$).

1 (Important) Forward elimination changes $\begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} x = b$ to a triangular $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} x = c$:

$$\begin{array}{l} x + y = 5 \\ x + 2y = 7 \end{array} \longrightarrow \begin{array}{l} x + y = 5 \\ y = 2 \end{array} \qquad \begin{bmatrix} 1 & 1 & 5 \\ 1 & 2 & 7 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 1 & 5 \\ 0 & 1 & 2 \end{bmatrix}$$

That step subtracted $\ell_{21} =$ _____ times row 1 from row 2. The reverse step *adds* ℓ_{21} times row 1 to row 2. The matrix for that reverse step is $L =$ _____. Multiply this L times the triangular system $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} x_1 = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$ to get _____ = _____. In letters, L multiplies $Ux = c$ to give _____.

2 Write down the 2 by 2 triangular systems $Lc = b$ and $Ux = c$ from Problem 1. Check that $c = (5,2)$ solves the first one. Find x that solves the second one.

3 (Move to 3 by 3) Forward elimination changes $Ax = b$ to a triangular $Ux = c$:

$$\begin{array}{lll} x + y + z = 5 & x + y + z = 5 & x + y + z = 5 \\ x + 2y + 3z = 7 & y + 2z = 2 & y + 2z = 2 \\ x + 3y + 6z = 11 & 2y + 5z = 6 & z = 2 \end{array}$$

The equation $z = 2$ in $Ux = c$ comes from the original $x + 3y + 6z = 11$ in $Ax = b$ by subtracting $\ell_{31} =$ _____ times equation 1 and $\ell_{32} =$ _____ times the *final* equation 2. Reverse that to recover $[1 \ 3 \ 6 \ 11]$ in the last row of A and b from the final $[1 \ 1 \ 1 \ 5]$ and $[0 \ 1 \ 2 \ 2]$ and $[0 \ 0 \ 1 \ 2]$ in U and c:

$$\text{Row 3 of } \begin{bmatrix} A & b \end{bmatrix} = (\ell_{31} \text{ Row 1} + \ell_{32} \text{ Row 2} + 1 \text{ Row 3}) \text{ of } \begin{bmatrix} U & c \end{bmatrix}.$$

In matrix notation this is multiplication by L. So $A = LU$ and $b = Lc$.

4 What are the 3 by 3 triangular systems $Lc = b$ and $Ux = c$ from Problem 3? Check that $c = (5, 2, 2)$ solves the first one. Which x solves the second one?

5 What matrix E puts A into triangular form $EA = U$? Multiply by $E^{-1} = L$ to factor A into LU:

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 4 & 2 \\ 6 & 3 & 5 \end{bmatrix}.$$

6 What two elimination matrices E_{21} and E_{32} put A into upper triangular form $E_{32}E_{21}A = U$? Multiply by E_{32}^{-1} and E_{21}^{-1} to factor A into $LU = E_{21}^{-1}E_{32}^{-1}U$:

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 5 \\ 0 & 4 & 0 \end{bmatrix}.$$

7 What three elimination matrices E_{21}, E_{31}, E_{32} put A into its upper triangular form $E_{32}E_{31}E_{21}A = U$? Multiply by E_{32}^{-1}, E_{31}^{-1} and E_{21}^{-1} to factor A into L times U:

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 2 & 2 & 2 \\ 3 & 4 & 5 \end{bmatrix} \qquad L = E_{21}^{-1}E_{31}^{-1}E_{32}^{-1}.$$

8 Suppose A is already lower triangular with 1's on the diagonal. Then $U = I$!

$$A = L = \begin{bmatrix} 1 & 0 & 0 \\ a & 1 & 0 \\ b & c & 1 \end{bmatrix}.$$

The elimination matrices E_{21}, E_{31}, E_{32} contain $-a$ then $-b$ then $-c$.

(a) Multiply $E_{32}E_{31}E_{21}$ to find the single matrix E that produces $EA = I$.

(b) Multiply $E_{21}^{-1}E_{31}^{-1}E_{32}^{-1}$ to bring back L (nicer than E).

9 *When zero appears in a pivot position, $A = LU$ is not possible!* (We are requiring nonzero pivots in U.) Show directly why these are both impossible:

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \ell & 1 \end{bmatrix} \begin{bmatrix} d & e \\ 0 & f \end{bmatrix} \qquad \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & & \\ \ell & 1 & \\ m & n & 1 \end{bmatrix} \begin{bmatrix} d & e & g \\ & f & h \\ & & i \end{bmatrix}.$$

This difficulty is fixed by a row exchange. That needs a "permutation" P.

10 Which number c leads to zero in the second pivot position? A row exchange is needed and $A = LU$ will not be possible. Which c produces zero in the third pivot position? Then a row exchange can't help and elimination fails:

$$A = \begin{bmatrix} 1 & c & 0 \\ 2 & 4 & 1 \\ 3 & 5 & 1 \end{bmatrix}.$$

11 What are L and D (the diagonal ***pivot matrix***) for this matrix A? What is U in $A = LU$ and what is the new U in $A = LDU$?

Already triangular
$$A = \begin{bmatrix} 2 & 4 & 8 \\ 0 & 3 & 9 \\ 0 & 0 & 7 \end{bmatrix}.$$

12 A and B are symmetric across the diagonal (because $4 = 4$). Find their triple factorizations LDU and say how U is related to L for these symmetric matrices:

Symmetric
$$A = \begin{bmatrix} 2 & 4 \\ 4 & 11 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 4 & 0 \\ 4 & 12 & 4 \\ 0 & 4 & 0 \end{bmatrix}.$$

13 (*Recommended*) Compute L and U for the symmetric matrix A:

$$A = \begin{bmatrix} a & a & a & a \\ a & b & b & b \\ a & b & c & c \\ a & b & c & d \end{bmatrix}.$$

Find four conditions on a, b, c, d to get $A = LU$ with four pivots.

14 This nonsymmetric matrix will have the same L as in Problem **13**:

Find L and U for
$$A = \begin{bmatrix} a & r & r & r \\ a & b & s & s \\ a & b & c & t \\ a & b & c & d \end{bmatrix}.$$

Find the four conditions on a, b, c, d, r, s, t to get $A = LU$ with four pivots.

Problems 15-16 use L and U (without needing A) to solve $Ax = b$.

15 Solve the triangular system $Lc = b$ to find c. Then solve $Ux = c$ to find x:

$$L = \begin{bmatrix} 1 & 0 \\ 4 & 1 \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} 2 & 4 \\ 0 & 1 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 2 \\ 11 \end{bmatrix}.$$

For safety multiply LU and solve $Ax = b$ as usual. Circle c when you see it.

16 Solve $Lc = b$ to find c. Then solve $Ux = c$ to find x. **What was A?**

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}.$$

17 (a) When you apply the usual elimination steps to L, what matrix do you reach?

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{bmatrix}.$$

(b) When you apply the same steps to I, what matrix do you get?

(c) When you apply the same steps to LU, what matrix do you get?

18 If $A = LDU$ and also $A = L_1D_1U_1$ with all factors invertible, then $L = L_1$ and $D = D_1$ and $U = U_1$. *"The three factors are unique."*

Derive the equation $L_1^{-1}LD = D_1U_1U^{-1}$. Are the two sides triangular or diagonal? Deduce $L = L_1$ and $U = U_1$ (they all have diagonal 1's). Then $D = D_1$.

19 *Tridiagonal matrices* have zero entries except on the main diagonal and the two adjacent diagonals. Factor these into $A = LU$ and $A = LDL^{\mathrm{T}}$:

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix} \quad \text{and} \quad A = \begin{bmatrix} a & a & 0 \\ a & a+b & b \\ 0 & b & b+c \end{bmatrix}.$$

20 When T is tridiagonal, its L and U factors have only two nonzero diagonals. How would you take advantage of knowing the zeros in T, in a code for Gaussian elimination? Find L and U.

Tridiagonal

$$T = \begin{bmatrix} 1 & 2 & 0 & 0 \\ 2 & 3 & 1 & 0 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 3 & 4 \end{bmatrix}.$$

21 If A and B have nonzeros in the positions marked by x, which zeros (marked by 0) *stay zero* in their factors L and U?

$$A = \begin{bmatrix} x & x & x & x \\ x & x & x & 0 \\ 0 & x & x & x \\ 0 & 0 & x & x \end{bmatrix} \qquad B = \begin{bmatrix} x & x & x & 0 \\ x & x & 0 & x \\ x & 0 & x & x \\ 0 & x & x & x \end{bmatrix}.$$

22 Suppose you eliminate upwards (almost unheard of). Use the last row to produce zeros in the last column (the pivot is 1). Then use the second row to produce zero above the second pivot. Find the factors in the unusual order $A = UL$.

Upper times lower
$$A = \begin{bmatrix} 5 & 3 & 1 \\ 3 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

23 *Easy but important.* If A has pivots 5, 9, 3 with no row exchanges, what are the pivots for the upper left 2 by 2 submatrix A_2 (without row 3 and column 3)?

Challenge Problems

24 Which invertible matrices allow $A = LU$ (elimination without row exchanges)? *Good question*! Look at each of the square upper left submatrices of A.

All upper left k by k submatrices A_k must be invertible **(sizes $k = 1, \ldots, n$).**

Explain that answer: A_k factors into _____ because $LU = \begin{bmatrix} L_k & 0 \\ * & * \end{bmatrix}\begin{bmatrix} U_k & * \\ 0 & * \end{bmatrix}$.

25 For the 6 by 6 second difference constant-diagonal matrix K, put the pivots and multipliers into $K = LU$. (L and U will have only two nonzero diagonals, because K has three.) Find a formula for the i, j entry of L^{-1}, by software like MATLAB using inv(L) or by looking for a nice pattern.

$-1, 2, -1$ matrix
$$K = \begin{bmatrix} 2 & -1 & & & & \\ -1 & \cdot & \cdot & & & \\ & \cdot & \cdot & \cdot & & \\ & & \cdot & \cdot & \cdot & \\ & & & \cdot & \cdot & -1 \\ & & & & -1 & 2 \end{bmatrix} = \text{toeplitz}([2 \ -1 \ 0 \ 0 \ 0 \ 0])$$

26 If you print K^{-1}, it doesn't look so good. But if you print $7K^{-1}$ (when K is 6 by 6), that matrix looks wonderful. Write down $7K^{-1}$ by hand, following this pattern:

1. Row 1 and column 1 are $(6, 5, 4, 3, 2, 1)$.
2. On and above the main diagonal, row i is i times row 1.
3. On and below the main diagonal, column j is j times column 1.

Multiply K times that $7K^{-1}$ to produce $7I$. Here is that pattern for $n = 3$:

3 by 3 case
The determinant of this K is 4
$$(K)(4K^{-1}) = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}\begin{bmatrix} 3 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 4 & & \\ & 4 & \\ & & 4 \end{bmatrix}.$$

2.7 Transposes and Permutations

We need one more matrix, and fortunately it is much simpler than the inverse. It is the *"transpose"* of A, which is denoted by A^T. *The columns of A^T are the rows of A.*

When A is an m by n matrix, the transpose is n by m:

Transpose If $A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 4 \end{bmatrix}$ then $A^T = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 3 & 4 \end{bmatrix}$.

You can write the rows of A into the columns of A^T. Or you can write the columns of A into the rows of A^T. The matrix "flips over" its main diagonal. The entry in row i, column j of A^T comes from row j, column i of the original A:

Exchange rows and columns $(A^T)_{ij} = A_{ji}$.

The transpose of a lower triangular matrix is upper triangular. (But the inverse is still lower triangular.) The transpose of A^T is A.

Note MATLAB's symbol for the transpose of A is A'. Typing [1 2 3] gives a row vector and the column vector is $v = [1\ 2\ 3]'$. To enter a matrix M with second column $w = [4\ 5\ 6]'$ you could define $M = [v\ w]$. Quicker to enter by rows and then transpose the whole matrix: $M = [1\ 2\ 3;\ 4\ 5\ 6]'$.

The rules for transposes are very direct. We can transpose $A + B$ to get $(A + B)^T$. Or we can transpose A and B separately, and then add $A^T + B^T$—with the same result. The serious questions are about the transpose of a product AB and an inverse A^{-1}:

Sum The transpose of $A + B$ is $A^T + B^T$. (1)

Product The transpose of AB is $(AB)^T = B^T A^T$. (2)

Inverse The transpose of A^{-1} is $(A^{-1})^T = (A^T)^{-1}$. (3)

Notice especially how $B^T A^T$ comes in reverse order. For inverses, this reverse order was quick to check: $B^{-1}A^{-1}$ times AB produces I. To understand $(AB)^T = B^T A^T$, start with $(Ax)^T = x^T A^T$:

Ax combines the columns of A while $x^T A^T$ combines the rows of A^T.

It is the same combination of the same vectors! In A they are columns, in A^T they are rows. So the transpose of the column Ax is the row $x^T A^T$. That fits our formula $(Ax)^T = x^T A^T$. Now we can prove the formula $(AB)^T = B^T A^T$, when B has several columns.

If $B = [x_1\ x_2]$ has two columns, apply the same idea to each column. The columns of AB are Ax_1 and Ax_2. Their transposes are the rows of $B^T A^T$:

$$\text{Transposing } AB = \begin{bmatrix} Ax_1 & Ax_2 & \cdots \end{bmatrix} \text{ gives } \begin{bmatrix} x_1^T A^T \\ x_2^T A^T \\ \vdots \end{bmatrix} \text{ which is } B^T A^T. \quad (4)$$